



REPUBLIC OF THE PHILIPPINES

POLYTECHNIC UNIVERSITY OF THE PHILIPPINES

OFFICE OF THE VICE PRESIDENT FOR ACADEMIC AFFAIRS

COLLEGE OF COMPUTER AND INFORMATION SCIENCES



Game Proposal: Zuma Extreme

Submitted by:

Group 13

Absalon, Rainier Aaron P.

Consolacion, Karizha C.

Gonzales, Eujen C.

Silvor, Cyrus Joseph C.

BSCS 3-4

April 14, 2026

Submitted to:

Prof. Ria A. Sagum

I. Introduction and its Background

Zuma is a 2003 tile-matching puzzle game where players control a frog that shoots colored balls from its mouth to create matches of three or more. It is popular for having an Aztec and Mayan themed environment, simple mechanics, and fast-paced gameplay.

The proposed game is inspired by Zuma but introduces a Player vs Environment (PvE) twist and transforms the popular childhood game into a turn-based system. In this version, players take turns shooting one ball at a time. Five balls of the same colors are also now required to be matched instead of three. Additionally, players can interfere with their opponent by breaking color combinations, adding a competitive element in this classic game.

By combining the classic tile-matching mechanics with turn-based gameplay and PVE, the game will offer a more competitive, strategic, and enjoyable experience compared to traditional puzzle games.

II. Game Proper

a. Objective of the Game

The goal is to be the last one standing. You win if your opponent's Health (HP) runs out first. You do this by making clever shots and forcing them to deal with the balls when they get too close to the end!

b. Discuss the Rule of the Game

- **Taking Turns:** You and your opponent take turns shooting one ball at a time. The line of balls moves a little bit after every shot.
- **Starting point:** Each player has 100 HP.
- **The Power of 5:** In regular Zuma, you need 3 balls to make them disappear. In this game, you need 5 balls of the same color to match.
- **Sabotage (Messing with your opponent):** Since it takes 5 balls to clear them, it's easy to see what your opponent is planning. If you see them building a group of 4 red balls, you can shoot a blue ball right in the middle to "mess up" their group!
- **Health (HP) and the Mouth:** * If the balls reach the "Mouth" at the end of the track, the player whose turn it is starts losing health.
 - The only way to stop losing health is to make a match of 5 balls. This clears the balls and pushes the whole line back to safety.
- **Winning condition:** The game ends when one of the players reaches 0 HP. The player that has the highest accumulated points wins the game.
- **Pointing System:**

Action	Points	How to Get
Basic Match	+50	Match exactly 5 balls.
Big Match	+100	Matched 6 or more balls at once.

Combo (Chain Reaction)	+150	One match caused another match to happen automatically.
Saving Yourself	+75	Clear balls while it's in the danger zone already.*

*Danger zone: When the ball starts to go inside the mouth.

III. Implementation of AI Algorithm

a. AI Algorithm Implemented in the Game

Tabular Q-Learning

This algorithm was chosen since it helps the AI adapt to the gameplay over time. As it plays, it learns from the outcomes of its own actions, such as what worked and what didn't, and applies that knowledge to make better decisions in future turns.

The AI learns through a reward system. It receives positive rewards for actions that bring it closer to winning, and negative rewards for actions that hurt its position. Over thousands of games, the AI will naturally discover that:

- Building clusters of 5 leads to good rewards.
- Sabotaging the opponent's clusters leads to good rewards.
- Letting the chain reach the danger zone leads to bad rewards.

Fuzzy Logic

This algorithm was chosen since it handles the vague, in-between states that are hard to define clearly. In Zuma Extreme, that means things like:

- How dangerous is the chain's current position?
- How close is a cluster to clearing?
- How at risk is the player's HP?

Instead of hard cutoffs like "danger zone starts at position 5", fuzzy logic might say "the chain is somewhat dangerous at position 7, very dangerous at position 5, and critically dangerous at position 3."

Pseudocode

INITIALIZE Q-table with all values set to 0

INITIALIZE fuzzy membership functions for:

- chain danger level (safe, moderate, critical)
- cluster progress (low, medium, high)
- HP status (healthy, moderate, critical)

FUNCTION fuzzify(state):

```
danger_level = evaluate chain position using fuzzy rules
cluster_progress = evaluate largest cluster using fuzzy rules
hp_status = evaluate current HP using fuzzy rules
RETURN fuzzy_state = (danger_level, cluster_progress, hp_status)
```

```
FOR each game in number_of_games:
```

```
    state = get_current_game_state()
    fuzzy_state = fuzzify(state)
```

```
    WHILE game is not over:
```

```
        IF random() < exploration_rate:
```

```
            action = random shot position
```

```
        ELSE:
```

```
            action = highest Q-value action for fuzzy_state
```

```
        new_state = apply_action(state, action)
```

```
        new_fuzzy_state = fuzzify(new_state)
```

```
        reward = get_reward(new_state)
```

```
        Q[fuzzy_state][action] = Q[fuzzy_state][action] + learning_rate *
            (reward + discount_factor *
            MAX(Q[new_fuzzy_state]) - Q[fuzzy_state][action])
```

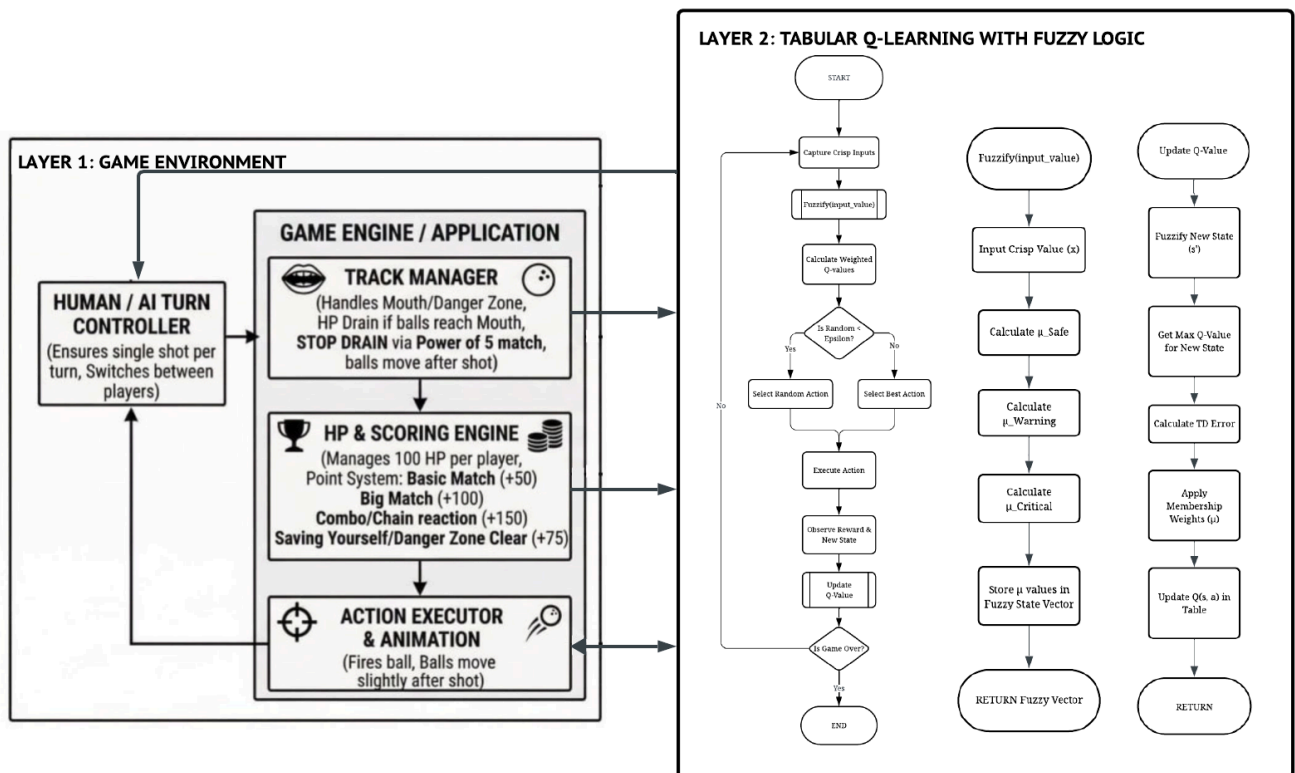
```
        state = new_state
```

```
        fuzzy_state = new_fuzzy_state
```

```
    exploration_rate = exploration_rate * exploration_decay
```

b. Software Architecture

This architecture is designed to decouple the Game Environment from the AI Intelligence, ensuring that the complex logic of Reinforcement Learning doesn't interfere with the real-time performance of the game engine. By implementing a modular "Layer 2," the AI can independently process raw game data into Fuzzy States, allowing it to handle the uncertainty of board positions more efficiently than a standard rule-based system. This separation also simplifies debugging, as you can optimize the Q-Learning memory and learning rates without needing to rewrite the core game mechanics or animation handlers.



Layer 1 and Layer 2 is a continuous cycle of data exchange that functions like a conversation between the "body" and "brain" of the system. Layer 1 serves as the sensory foundation by providing raw data, often called crisp inputs, which describe the current state of the game such as exact ball coordinates and current HP levels. These numbers are transmitted to Layer 2, which performs discretization, the process of turning continuous, precise data into distinct categories. Using fuzzy logic, Layer 2 translates these raw numbers into membership values, allowing the AI to understand the board not as a series of coordinates, but as "shades of truth" like being 80% in the "Danger" zone and 20% in the "Warning" zone.

Once the AI has processed this state, it selects an action and sends the command back to Layer 1 for physical execution. Layer 1 fires the ball, calculates the physics of the "Power of 5" matches, and monitors for any stochastic events, which are occurrences involving an element of chance or randomness, such as the color of the next ball in the queue. After the action is complete, Layer 1 sends a reward, a numerical value based on your points system, back to Layer 2. This creates a feedback loop where the AI uses the result to update its internal memory, or Q-table. By constantly comparing the expected outcome with the actual reward received from Layer 1, Layer 2 refines its strategy to better handle sabotage and HP management in subsequent turns.